

Circular Linked Lists & Linked List in Array

Representation, Traversal, Insertion, Deletion and Cursor Implementation

Dr. Siddhant Thapliyal

Department of Computer Science and Engineering
Graphic Era (Deemed to be University), Dehradun

Quick Recall: Singly Linked List

In a normal singly linked list:

- Each node contains DATA and NEXT.
- START points to the first node.
- NEXT points to the succeeding node.
- The last node contains:

NEXT = NULL

Traversal therefore stops when:

PTR = NULL

START



Linear Linked List

What If the Last Node Points Back?

Normal linked list:

$10 \rightarrow 20 \rightarrow 30 \rightarrow \text{NULL}$

Now replace:

$30 \rightarrow \text{NULL}$

with:

$30 \rightarrow 10$

Circular Linked List

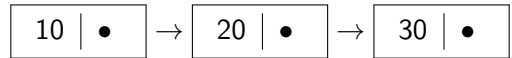
A **circular linked list** is a linked list in which the last node points back to the first node.
Therefore:

$$LAST \rightarrow NEXT = START$$

Important properties:

- There is no NULL link at the end.
- Nodes form a closed loop.
- Traversal can begin from any node.
- We must explicitly decide when traversal should stop

START



Linear vs Circular Linked List

Linear Singly List	Circular Singly List
Last node points to NULL	Last node points to START
Has a definite end	Nodes form a loop
Traversal ends at NULL	Traversal ends on returning to START
Easy stopping condition	Care needed to avoid infinite loop
Useful for linear sequences	Useful for cyclic processes

NULL

vs

Return to START

Structure of a Circular List

Suppose the list contains:

10, 20, 30

The links are:

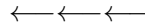
10.*next* → 20

20.*next* → 30

30.*next* → 10

Therefore:

30.*next* = *START*



Circular List in Memory

Consider:

$$START = 100$$

Address	DATA	NEXT
100	10	500
500	20	900
900	30	100

Logical representation:

100 → 500 → 900 → 100

In a circular linked list:

$$LAST \rightarrow NEXT = START$$

Not:

$$LAST \rightarrow NEXT = NULL$$

Where Are Circular Lists Useful?

Circular structures are useful when processing has to repeat continuously.

Examples:

- Round-robin CPU scheduling
- Repeated task management
- Circular buffers
- Multiplayer turn management
- Playlist repetition
- Token passing systems

$P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4$

↑

↓

←←←

After P_4 , processing returns to P_1 .

Node Representation in C

The node structure is exactly the same as a normal singly linked list:

```
struct Node
{
    int data;
    struct Node *next;
};
```

The difference is not in the structure definition.

The difference is in the **links**:

$$last \rightarrow next = start$$

rather than:

$$last \rightarrow next = NULL$$

A One-Node Circular List

Suppose the first node contains 10.
For a normal singly linked list:

$newNode \rightarrow next = NULL$

For a circular list:

$newNode \rightarrow next = newNode$

and:

$START = newNode$

START



10



Creating the First Circular Node

```
1 struct Node *newNode;  
2  
3 newNode =  
4     (struct Node *)malloc(sizeof(struct Node));  
5  
6 newNode->data = 10;  
7  
8 newNode->next = newNode;  
9  
0 start = newNode;
```

START →

10		START
----	--	-------

Traversal of a Circular Linked List

In a linear list we stop when:

$$PTR = NULL$$

But a circular list never reaches NULL.

Therefore traversal must stop when the pointer returns to START.

Typical condition:

$$PTR \rightarrow next = START$$

or using a do-while loop:

$$PTR \neq START$$

after processing at least one node

$$START \rightarrow 10 \rightarrow 20 \rightarrow 30$$

$$30 \rightarrow START$$

Sequence:

$$10, 20, 30, STOP$$

not:

Traversal: Dry Run

$START \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow START$

Step	PTR points to	Action
1	10	Print 10
2	20	Print 20
3	30	Print 30
4	START again	Stop

Output:

10 20 30

Traversal Using do-while

```
void display(struct Node *start)
{
    if (start == NULL)
    {
        printf("List is empty\n");
        return;
    }

    struct Node *ptr = start;

    do
    {
        printf("%d ", ptr->data);
        ptr = ptr->next;
    } while (ptr != start);
}
```

Why is do-while Convenient?

If the list is non-empty, *START* already points to a valid node.

We must process that node before checking whether we returned to *START*.

Hence:

do-while

fits naturally.

General pattern:

Process → *Move* → *Check*

START



10 → 20 → 30



How Do We Find the Last Node?

For a linear list:

$$PTR \rightarrow next = NULL$$

identifies the last node.

For a circular list:

$$PTR \rightarrow next = START$$

identifies the last node.

Therefore:

$while(ptr \rightarrow next \neq start)$

moves PTR to the last node.

$$START \rightarrow 10 \rightarrow 20 \rightarrow \underbrace{30}_{PTR}$$

$$30.next = START$$

30 is last

Insertion Cases

For this lecture we consider two important cases:

- 1 Insert at the beginning.
- 2 Insert at the end.

Original:

$10 \rightarrow 20 \rightarrow 30 \rightarrow \textit{START}$

Possible results:

$5 \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow \textit{START}$

or

Insert at Beginning

Suppose we insert 5.

We need the last node because its NEXT currently points to START.

Steps:

① Create the new node.

② Find the last node.

③ Set:

$newNode \rightarrow next = start$

④ Set:

$last \rightarrow next = newNode$

⑤ Update:

$start = newNode$

Before

10 → 20 → 30

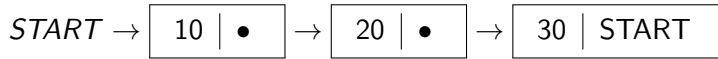
30 → 10

After

5 → 10 → 20 → 30

Insertion at Beginning: Pointer Changes

Before:



New node:



Change last node:

$30.next = NEW$

and:

Insert at Beginning: C Code

```
struct Node *insertBeginning(  
    struct Node *start, int value)  
{  
    struct Node *newNode, *ptr;  
  
    newNode = malloc(sizeof(struct Node));  
  
    newNode->data = value;  
  
    if (start == NULL)  
    {  
        newNode->next = newNode;  
        return newNode;  
    }  
  
    ptr = start;
```

Insert at End

Suppose we insert 40 at the end.

Steps:

- 1 Allocate a new node.
- 2 Find the last node.
- 3 Connect:

$last \rightarrow next = newNode$

- 4 Make the new node point to START:

$newNode \rightarrow next = start$

START does not change.

Before

10 → 20 → 30

30 → 10

After

10 → 20 → 30 → 40

Insert at End: C Code

```
struct Node *insertEnd(  
    struct Node *start, int value)  
{  
    struct Node *newNode, *ptr;  
  
    newNode = malloc(sizeof(struct Node));  
    newNode->data = value;  
  
    if (start == NULL)  
    {  
        newNode->next = newNode;  
        return newNode;  
    }  
  
    ptr = start;  
  
    while (ptr->next != start)
```

Insertion Complexity

Operation	Without Tail	With Tail
Insert at beginning	$O(n)$	$O(1)$
Insert at end	$O(n)$	$O(1)$

Why?

Without a tail pointer, we must find the last node:

```
while(ptr->next != start)
```

A separate tail pointer avoids this traversal.

Deletion Cases

We now study:

- 1 Delete the first node.
- 2 Delete the last node.

Two special cases must always be remembered:

- Empty list.
- List containing only one node.

$$START = NULL$$

or:

$$START \rightarrow next = START$$

Delete First Node

Suppose the list is:

10 → 20 → 30 → 10

To delete 10:

- 1 Find the last node.
- 2 Save old START.
- 3 Move START to second node.
- 4 Make last node point to new START.
- 5 Free the old first node.

Before

10 → 20 → 30

30 → 10

After

20 → 30

Deletion at Beginning: Key Pointer Changes

Suppose PTR is the last node.

$$TEMP = START$$
$$START = START \rightarrow next$$
$$PTR \rightarrow next = START$$
$$free(TEMP)$$

Before:

Delete Beginning: C Code

```
struct Node *deleteBeginning(  
    struct Node *start)  
{  
    struct Node *last, *temp;  
  
    if (start == NULL)  
        return NULL;  
  
    if (start->next == start)  
    {  
        free(start);  
        return NULL;  
    }  
  
    last = start;  
  
    while (last->next != start)
```

Delete Last Node

We need two pointers:

PTR

for the current node, and:

PREPTR

for the preceding node.

When *PTR* becomes the last node:

$PTR \rightarrow next = START$

we update:

$PREPTR \rightarrow next = START$

Before:

10 → 20 → 30
PTR

30 → 10

After:

10 → 20
PREPTR

Delete End: C Code

```
1 struct Node *deleteEnd(struct Node *start)
2 {
3     struct Node *ptr, *preptr;
4
5     if (start == NULL)
6         return NULL;
7
8     if (start->next == start)
9     {
10         free(start);
11         return NULL;
12     }
13
14     ptr = start;
15
16     while (ptr->next != start)
17     {
```

Single-Node Case

Suppose:

$START \rightarrow 10$

and:

$10.next = START$

Then:

$START \rightarrow next = START$

indicates a one-node circular list.

Deleting this node requires:

$free(START)$

followed by:

Before:

$START \rightarrow \boxed{10}$

↑ ↓

←

After:

Common Programming Error

Incorrect traversal:

```
while(ptr! = NULL)
```

Why is this dangerous?

Common Programming Error

Incorrect traversal:

```
while(ptr! = NULL)
```

Why is this dangerous?

Because:

PTR

will never naturally become NULL.

It continues:

Complete Program: Structure and Display

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 void display(struct Node *start)
11 {
12     struct Node *ptr;
13
14     if (start == NULL)
15     {
16         printf("List is empty\n");
17         return;
18     }
19 }
```

Circular List: Main Function

```
int main()
{
    struct Node *start = NULL;

    start = insertEnd(start, 10);
    start = insertEnd(start, 20);
    start = insertEnd(start, 30);

    printf("Initial List:\n");
    display(start);

    start = insertBeginning(start, 5);

    printf("\nAfter inserting 5:\n");
    display(start);

    start = deleteEnd(start);
```

Expected Output

Initial List:

10 -> 20 -> 30 -> START

After inserting 5:

5 -> 10 -> 20 -> 30 -> START

After deleting last node:

5 -> 10 -> 20 -> START

Notice:

START

appears instead of NULL.

Linked List Without Pointers?

So far:

NEXT

stores a memory address.

But what if we store:

ARRAY INDEX

instead?

Linked List in an Array

Instead of storing:

Address of next node

we store:

Index of next element

Each array element contains:

<i>DATA</i> <i>NEXT</i>

where NEXT is an integer index.

Example:

START = 2

Index	DATA	NEXT
0	–	–
1	30	-1
2	10	4
3	–	–
4	20	1

Follow the Indices

Given:

$$START = 2$$

and:

Index	DATA	NEXT
1	30	-1
2	10	4
4	20	1

Traversal:

$$2 \rightarrow 4 \rightarrow 1 \rightarrow -1$$

Therefore DATA sequence is:

$$10 \rightarrow 20 \rightarrow 30$$

Physical vs Logical Order

Physical Array

1	30
2	10
3	unused
4	20

The elements are not logically ordered by their array positions.

Logical List

$START = 2$

$2 \rightarrow 4 \rightarrow 1 \rightarrow -1$

Therefore:

$10 \rightarrow 20 \rightarrow 30$

C Representation

A cursor-based node may be represented as:

```
struct ArrayNode
{
    int data;
    int next;
};

struct ArrayNode list[10];
```

For example:

```
list[2].data = 10;
list[2].next = 4;

list[4].data = 20;
list[4].next = 1;
```

Traversing a Linked List Stored in an Array

```
void displayArrayList(  
    struct ArrayNode list[], int start)  
{  
    int ptr = start;  
  
    while (ptr != -1)  
    {  
        printf("%d ", list[ptr].data);  
        ptr = list[ptr].next;  
    }  
}
```

Instead of:

$$ptr = ptr \rightarrow next$$

we use:

Array-Based List: Dry Run

Index	DATA	NEXT
0	50	-1
1	40	0
2	10	4
3	30	1
4	20	3

Given:

$START = 2$

Follow NEXT:

$2 \rightarrow 4 \rightarrow 3 \rightarrow 1 \rightarrow 0 \rightarrow -1$

Therefore:

Why Represent a Linked List in an Array?

- Useful when pointer operations are unavailable or undesirable.
- Useful for teaching logical links separately from physical addresses.
- Can provide predictable, fixed memory usage.
- Array indices act as “pointers”.
- Can be useful in embedded or memory-constrained implementations.

Pointer Address $\longleftrightarrow$ Array Index

Limitations of Array-Based Linked Lists

Advantages

- No actual pointer arithmetic.
- Easy serialization.
- Fixed memory region.
- Indices are easy to inspect.

Limitations

- Maximum size must usually be fixed.
- Free locations must be managed.
- Less naturally dynamic than pointer-based lists.
- Extra logic is required for allocation.

How Do We Manage Free Array Locations?

Unused locations can themselves form another linked list.

A variable such as:

AVAIL

stores the index of the first free location.

Example:

$AVAIL = 3$

Free-chain:

$3 \rightarrow 5 \rightarrow 7 \rightarrow -1$

When a node is needed, an index is removed from *AVAIL*.

Index	NEXT Free
3	5
5	7
7	-1

$AVAIL = 3$

Three Representations Compared

Array	Pointer List	Array Linked List
Physical order gives sequence Direct indexing	Pointers give sequence Sequential traversal	Indices give sequence Sequential traversal
Contiguous storage No NEXT field	Dynamic scattered storage Pointer NEXT	Fixed array storage Integer NEXT index

Complexity Recap

Operation	Circular List	Array-Linked List
Traversal	$O(n)$	$O(n)$
Search	$O(n)$	$O(n)$
Insert beginning*	$O(n)$	Depends on representation
Insert end*	$O(n)$	Depends on representation

*For circular lists without maintaining a separate tail pointer.

Logical traversal is still sequential

Classroom Activity 1

Consider:

START $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ *START*

Perform:

- 1 Insert 5 at the beginning.
- 2 Insert 40 at the end.
- 3 Delete the first node.
- 4 Delete the last node.

Draw the circular connection after every operation.

Classroom Activity 1: Answer

Initial:

$10 \rightarrow 20 \rightarrow 30 \rightarrow 10$

Insert 5:

$5 \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow 5$

Insert 40:

$5 \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow 5$

Delete first:

$10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow 10$

Delete last:

$10 \rightarrow 20 \rightarrow 30 \rightarrow 10$

Classroom Activity 2

Given:

Index	DATA	NEXT
0	15	3
1	50	-1
2	10	0
3	30	1

and:

$$START = 2$$

What is the logical linked list?

Classroom Activity 2

Given:

Index	DATA	NEXT
0	15	3
1	50	-1
2	10	0
3	30	1

and:

$$START = 2$$

What is the logical linked list?

$$2 \rightarrow 0 \rightarrow 3 \rightarrow 1 \rightarrow -1$$

Therefore:

MCQ 1

In a circular singly linked list, the NEXT field of the last node contains:

- ① NULL
- ② Address of START
- ③ Address of itself in every case
- ④ Zero only

MCQ 1

In a circular singly linked list, the NEXT field of the last node contains:

- ① NULL
- ② Address of START
- ③ Address of itself in every case
- ④ Zero only

Address of START

Which condition commonly identifies the last node of a circular linked list?

- ① `ptr == NULL`
- ② `ptr->next == NULL`
- ③ `ptr->next == start`
- ④ `ptr == -1`

Which condition commonly identifies the last node of a circular linked list?

- ① `ptr == NULL`
- ② `ptr->next == NULL`
- ③ `ptr->next == start`
- ④ `ptr == -1`

`ptr->next == start`

In an array-based linked list, NEXT usually stores:

- ① A floating-point value
- ② An array index
- ③ Always NULL
- ④ The DATA value

In an array-based linked list, NEXT usually stores:

- ① A floating-point value
- ② An array index
- ③ Always NULL
- ④ The DATA value

Array Index

Find the Error

```
ptr = start;

while (ptr != NULL)
{
    printf("%d ", ptr->data);
    ptr = ptr->next;
}
```

Why is this incorrect for a non-empty circular linked list?

Find the Error

```
ptr = start;

while (ptr != NULL)
{
    printf("%d ", ptr->data);
    ptr = ptr->next;
}
```

Why is this incorrect for a non-empty circular linked list?

Because:

ptr may never become NULL

The program may run forever.

Quick Recap

Circular Linked List

$$LAST \rightarrow NEXT = START$$

Traversal:

$$START \rightarrow \dots \rightarrow START$$

No NULL at the end.

Good for cyclic processing.

Linked List in Array

$$NEXT = \text{index}$$

Traversal:

$$\text{index} \rightarrow \text{index} \rightarrow \dots$$

A special value such as -1 marks the end.

Practice Questions

- 1 Write a C program to create and display a circular linked list.
- 2 Insert a node at the beginning of a circular list.
- 3 Insert a node at the end.
- 4 Delete the first node.
- 5 Delete the last node.
- 6 Explain why `while(ptr != NULL)` is inappropriate.
- 7 Represent the list $10 \rightarrow 20 \rightarrow 30$ using an array.
- 8 Write a traversal function for an array-based linked list.

- ① Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*.
- ② Reema Thareja, *Data Structures Using C*.
- ③ Horowitz, Sahni and Anderson-Freed, *Fundamentals of Data Structures in C*.

Your Feedback Matters

Please share your feedback about:

- Circular-list visualization
- Traversal logic
- Insertion and deletion
- Special cases
- Linked-list-in-array concept
- C programming examples



Scan to Submit Feedback

Thank You

Questions?

$START \rightarrow [10] \rightarrow [20] \rightarrow [30] \rightarrow START$